

110.9. Devant, derrière : cache-toi du garde

Un petit jeu : traverse l'écran sans te faire voir. Le garde passe en haut ; derrière un arbre marqué DEVANT, il ne te voit pas.

1. Ce qu'il faut comprendre

Un petit jeu avec tout ce qu'on vient de voir. Ton héros part à gauche et doit atteindre la droite. Un garde fait les cent pas en haut de l'écran. S'il passe au-dessus de toi pendant que tu es à découvert, il te voit : retour au départ.

Les arbres sont des cachettes : leurs cases ont `DEVANT`. Derrière un arbre, ton héros disparaît, et le garde passe sans te voir.

Mais la console ne sait pas que tu es caché ! `DEVANT` ne change que le **dessin**. Pour le jeu, il faut le dire aussi dans le code : `cache` vaut 1 si le héros (8 pixels) est **entièrement** derrière un arbre (16 pixels). Le premier arbre va du pixel 40 au 55 : le héros y est caché pour `x` de 40 à 48 ($48 + 8 = 56$).

L'écart entre le garde et toi : `ecart` est la distance, toujours positive. Comme `uint8_t` ne connaît pas les nombres négatifs, on soustrait le plus petit du plus grand : `g - x` si le garde est à droite, `x - g` sinon. Moins de 12 pixels, et pas caché : vu !

Vu : le compteur `vus` augmente, le héros repart du pixel 8, et le garde repart de la droite (sinon il te reverrait aussitôt).

À toi : ajoute un troisième arbre, ou un second garde, plus rapide, en lutin 2.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <couleurFond>` → [0.88.3. La fonction couleurFond\(\) — une couleur du décor](#)
- `#include <couleurLutin>` → [0.107.4. La fonction couleurLutin\(\) — une couleur des personnages](#)
- `#include <ecran>` → [27.1. La fonction ecran\(\) — éteindre et rallumer l'écran](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <teindre>` → [0.89.1. La fonction teindre\(\) — une case dans une palette](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)
- `#include <teindreLutin>` → [0.107.5. La fonction teindreLutin\(\) — un lutin dans une palette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1333 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	71 o	← la console, à l'allumage – la mise en route de la console
main()	636 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← ecran() – attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre – attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), teindre() – trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() – écrit un nombre en chiffres
RangerLutins	11 o	← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles	94 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	320 o	← CopierTuiles – 17 lettres sur 43, et 3 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert – la petite routine qui copie les lutins
"VUS"	3 o	← texte() – un texte gravé
"GAGNE"	5 o	← texte() – un texte gravé

Lettres gravées : AEGNSUV0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

teindre()	x 4	92 o sur place, dans main() + AdresseCase (10 o)
couleurLutin()	x 6	72 o sur place, dans main()
couleurFond()	x 5	60 o sur place, dans main()
poser()	x 4	48 o sur place, dans main() + AdresseCase (10 o)
sprite()	x 2	46 o sur place, dans main()
teindreLutin()	x 2	33 o sur place, dans main()
texte()	x 2	22 o sur place, dans main() + EcrireTexte (10 o)
ecran()	x 2	21 o sur place, dans main() + AttendreVBlank (16 o)
nombre()	x 2	20 o sur place, dans main() + EcrireNombre (33 o)
bouton()	x 2	10 o sur place, dans main() + LireManette (37 o)
image()	x 1	6 o sur place, dans main() + AttendreImage (28 o)
images()	x 1	2 o sur place, dans main()

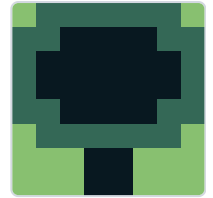
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME (Game Boy Color) : un petit jeu de cache-cache.
// GAUCHE / DROITE : ton héros (en bas) doit atteindre la droite de l'écran.
// Le GARDE fait les cent pas en haut. S'il passe au-dessus de toi pendant que
// tu es à découvert : VU, retour au départ.
// Derrière un ARBRE (cases DEVANT), tu es caché : il ne te voit pas.
//
// Ce qui est nouveau : DEVANT ne change que le DESSIN. Pour le jeu, on calcule
// aussi, dans le code, si le héros est caché (la variable cache).

#include <Tuile>           // un dessin de 8 x 8 pixels
#include <couleurFond>     // choisit une couleur d'une palette du fond
#include <couleurLutin>    // choisit une couleur d'une palette des lutins
#include <ecran>           // éteint ou rallume l'écran
#include <poser>           // pose une tuile sur une case du fond
#include <teindre>         // met une case du fond dans une palette
#include <texte>           // écrit un texte à l'écran
#include <nombre>          // écrit un nombre en chiffres
#include <bouton>          // lit un bouton de la manette
#include <sprite>          // place un lutin de 8 x 8 au pixel près
#include <teindreLutin>    // met un lutin dans une palette
```

2. UN DESSIN — ARBRE

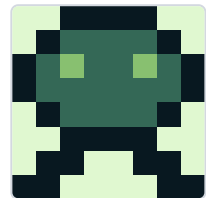
```
Tuile ARBRE = { // Du feuillage en haut, un tronc en bas. Aucun 0 : tout
plein.
    "12222221",
    "22333322",
    "23333332",
    "23333332",
    "22333322",
    "12222221",
    "11133111",
    "11133111",
};
```



à l'écran

3. UN DESSIN — HEROS

```
Tuile HEROS = { // Toi : 0 transparent, 1 clair, 2 moyen, 3 contour.
    "00333300",
    "03222230",
    "32122123",
    "32222223",
    "03222230",
    "00333300",
    "03300330",
    "33000033",
};
```



à l'écran

4. UN DESSIN — GARDE

```
Tuile GARDE = {                                // Le garde : un casque, de gros yeux.
    "03333330",
    "33333333",
    "31133113",
    "31133113",
    "32222223",
    "03222230",
    "03300330",
    "3300033",
};
```



à l'écran

5. UNE VARIABLE — X

```
uint8_t x = 8;           // Ta place, en pixels.
```

6. UNE VARIABLE — G

```
uint8_t g = 150;           // La place du garde.
```

7. UNE VARIABLE — SENS

```
uint8_t sens = 0; // Le garde va : 0 vers la gauche, 1 vers la droite.
```

8. UNE VARIABLE — VUS

```
uint8_t vus = 0; // Combien de fois il t'a vu.
```

9. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    couleurFond(0, 0, 20, 28, 31);          // palette 0 : le ciel et le texte
    couleurFond(0, 3, 2, 4, 12);
    couleurFond(2, 1, 8, 18, 4);            // palette 2 : l'arbre, vert,
    couleurFond(2, 2, 4, 26, 6);           // vert vif,
    couleurFond(2, 3, 0, 10, 2);           // vert sombre.
    couleurLutin(0, 1, 28, 31, 31);         // toi : bleu pâle,
    couleurLutin(0, 2, 4, 12, 31);         // bleu,
    couleurLutin(0, 3, 0, 0, 10);          // bleu nuit.
    couleurLutin(1, 1, 31, 31, 31);        // le garde : blanc,
    couleurLutin(1, 2, 31, 4, 2);          // rouge,
    couleurLutin(1, 3, 10, 0, 0);          // rouge sombre.

    ecran(0);                              // Dessin écran éteint.
    texte(1, 1, "VUS");
    // Deux arbres de 2 x 2 cases, lignes 8 et 9 : colonnes 5-6, puis 13-14.
    for (uint8_t l = 8; l < 10; l++) {
        poser(5, l, ARBRE); teindre(5, l, 2 | DEVANT);
        poser(6, l, ARBRE); teindre(6, l, 2 | DEVANT);
        poser(13, l, ARBRE); teindre(13, l, 2 | DEVANT);
        poser(14, l, ARBRE); teindre(14, l, 2 | DEVANT);
    }
    ecran(1);                              // On rallume.
    nombre(5, 1, vus);

    while (true) {                          // La boucle du jeu :
        image();

        // Toi : GAUCHE / DROITE.
        if (bouton(DROITE) && x < 152) x++;
        if (bouton(GAUCHE) && x > 8) x--;

        // Le garde : un pixel toutes les 2 images, et demi-tour aux bords.
        if (images() % 2 == 0) {
            if (sens == 0) {
                g--;
                if (g == 0) sens = 1;        // au bord gauche : il repart à droite
            } else {
                g++;
                if (g == 152) sens = 0;      // au bord droit : il repart à gauche
            }
        }

        // Caché ? Tout entier derrière un arbre (le héros fait 8 pixels, l'arbre 16).
        uint8_t cache = 0;
        if (x >= 40 && x <= 48) cache = 1;  // arbre 1 : pixels 40 à 55
        if (x >= 104 && x <= 112) cache = 1; // arbre 2 : pixels 104 à 119

        // L'écart entre le garde et toi, toujours positif.
        uint8_t ecart = 0;
        if (g > x) {
            ecart = g - x;                  // le garde est à droite
        } else {
            ecart = x - g;                  // le garde est à gauche (ou pile au-dessus)
        }

        if (ecart < 12 && cache == 0) {     // Tout près, et à découvert : VU !
            vus++;
            nombre(5, 1, vus);
            x = 8;                          // retour au départ,
            g = 150;                         // et le garde repart de la droite.
            sens = 0;
        }
    }
}
```

```

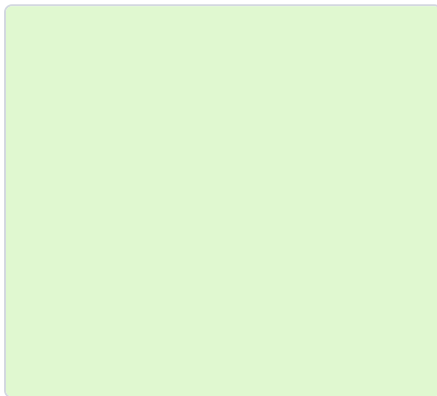
if (x >= 150) texte(8, 1, "GAGNE"); // Arrivé à droite !

sprite(0, x, 68, HEROS);           // Toi, en bas, à la hauteur des arbres.
teindreLutin(0, 0);
sprite(1, g, 40, GARDE);           // Le garde, en haut.
teindreLutin(1, 1);
}
}

```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



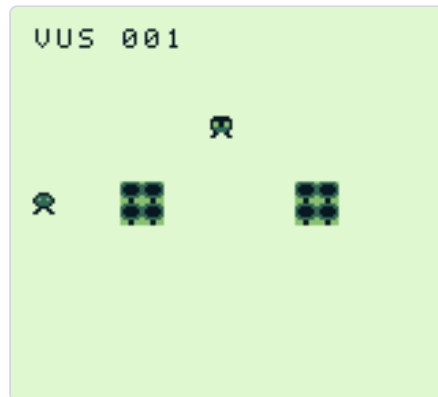
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



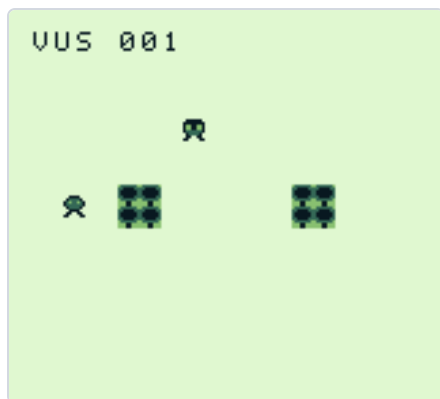
au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Deux arbres verts, ton héros bleu à gauche, un garde rouge qui va et vient en haut. Derrière un arbre, ton héros disparaît et le garde ne le voit pas ; à découvert, il te renvoie au départ.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ à découvert, le garde te voit (vus 1)

✓ derrière l'arbre, on voit l'arbre, pas toi (x 45 : 8/18/4)

✓ caché, le garde passe sans te voir (vus 1 puis 1)

Ensuite : **110.10. La fonction `spriteDerriere()` — un lutin derrière le décor** — chapitre 17 : Le mouvement : les lutins au pixel près. · gameboy3 — écrire une cartouche Game Boy en C++